

DBTable: Leveraging Discriminative Bitsets for High-Performance Packet Classification

Zhengyu Liao^{ID}, Shiyu Qian^{ID}, *Member, IEEE*, Zhonglong Zheng^{ID}, *Member, IEEE*, Jiange Zhang^{ID}, Jian Cao^{ID}, *Senior Member, IEEE*, Guangtao Xue^{ID}, *Member, IEEE*, and Minglu Li^{ID}, *Fellow, IEEE*

Abstract—Packet classification, as a crucial function of networks, has been extensively investigated. In recent years, the rapid advancement of software-defined networking (SDN) has introduced new demands for packet classification, particularly in supporting dynamic rule updates and fast lookup. This paper presents a novel structure called DBTable for efficient packet classification to achieve high overall performance. DBTable integrates the strengths of conventional packet classification methods and neural network concepts. Within DBTable, a straightforward indexing scheme is proposed to eliminate rule replication, thereby ensuring high update performance. Additionally, we propose an iterative method for generating a discriminative bitset (DBS) to evenly partition rules. By utilizing the DBS, rules can be efficiently mapped in a hash table, thus achieving exceptional lookup performance. Moreover, DBTable incorporates a hybrid structure to further optimize the worst-case lookup performance, primarily caused by data skewness. The experiment results on 12 256k rulesets show that, compared to seven state-of-the-art schemes, DBTable achieves an overall lookup speed improvement ranging from 1.53x to 7.29x, while maintaining the fastest update speed.

Index Terms—Packet classification, ruleset, index, performance.

I. INTRODUCTION

PACKET classification plays a fundamental role in a diverse range of Internet applications, including security, monitoring, and multimedia communications [1], [2], [3]. The prime objective of packet classification is to efficiently determine the appropriate flow for packet forwarding by searching through a set of predefined rules. However, the emergence of novel networking paradigms like software-defined networking (SDN) [4], [5], [6], [7], has introduced new challenges, such as a larger ruleset size [8] and the need for fast incremental updates. Additionally, the advancement of transmission technologies [9], [10] imposes higher wire speed requirements

for future packet classification. Consequently, this study aims to investigate a robust packet classification algorithm that achieves high overall performance in the context of the evolving Internet.

Over the past two decades, numerous research efforts have emerged to enhance the efficiency of packet classification. Hardware-based approaches, predominantly relying on ternary content addressable memory (TCAM), have demonstrated exceptional lookup performance in traditional networking [11], [12], [13], [14], [15], [16], [17], [18]. Nevertheless, due to their high cost and limited scalability to accommodate the expanding rulesets, researchers have increasingly focused on developing software-based solutions in recent years.

Algorithm-based software solutions can be divided into four categories: decision tree-based [19], [20], [21], [22], [23], [24], [25], [26], [27], [28], [29], [30], tuple space search-based [31], [32], [33], [34], neural network-based [35], [36], [37], and hybrid methods [38], [39], [40]. Decision tree-based methods aim to partition the search space into multiple subspaces. Despite their ability to achieve fast lookup speeds, they face challenges with rule replication during construction and updates. The Tuple Space Search (TSS) [31] scheme is specifically designed to fulfill the requirements of SDNs, such as OpenFlow [41]. By employing hash tables constructed based on the length of specified bits in rules, TSS only requires a single memory access for rule updating. In contrast to conventional approaches, neural network-based methods leverage machine learning techniques to optimize or transform the indexing structure for improved performance. Furthermore, several hybrid methods have been proposed to strike a balance between lookup performance and update performance by combining the strengths of decision tree and TSS methods. However, achieving efficient lookup and update performance simultaneously remains a challenge for these hybrid methods.

To obtain a high overall performance, we integrate the strengths of conventional methods and neural network-based approaches. The core concept revolves around partitioning the ruleset based on a discriminative bitset (DBS) derived from the ruleset. The objective is to achieve constant-time ($O(1)$) memory access for both lookup and update operations. To fulfill this objective, we propose a succinct index structure named DBTable, which constructs a hash table based on the DBS. By eliminating rule replication, DBTable ensures superior update performance. Moreover, to achieve rapid lookup and minimal storage requirements, we devise an iterative technique for extracting the DBS, which serves as the foundation

Manuscript received 6 January 2024; revised 29 July 2024; accepted 24 August 2024; approved by IEEE/ACM TRANSACTIONS ON NETWORKING Editor S. Schmid. Date of publication 10 September 2024; date of current version 19 December 2024. This work was supported in part by the National Natural Science Foundation of China under Grant 61772334 and in part by Huawei. (Corresponding authors: Shiyu Qian; Zhonglong Zheng.)

Zhengyu Liao, Zhonglong Zheng, and Minglu Li are with the School of Computer Science and Technology, Zhejiang Normal University, Zhejiang 321004, China (e-mail: zhengyu@zjnu.edu.cn; zhonglong@zjnu.edu.cn; mlli@zjnu.edu.cn).

Shiyu Qian, Jian Cao, and Guangtao Xue are with the Department of Computer Science and Engineering and Shanghai Key Laboratory of Trusted Data Circulation and Governance in Web3, Shanghai Jiao Tong University, Shanghai 200240, China (e-mail: qshiyu@sjtu.edu.cn; cao-jian@sjtu.edu.cn; gt_xue@sjtu.edu.cn).

Jiange Zhang is with Huawei Technologies Company Ltd., Beijing 100015, China (e-mail: jiange.zhang@huawei.com).

Digital Object Identifier 10.1109/TNET.2024.3452780

for evenly partitioning the ruleset. Additionally, to guarantee the worst-case lookup performance, DBTable incorporates a skewness-aware hybrid structure, enabling high-performance outcomes across diverse ruleset types.

We conduct a comprehensive evaluation of DBTable's performance in terms of lookup, update, construction, and memory footprint. This evaluation is carried out on 12 rulesets generated using Classbench-ng [42]. The size of these rulesets ranges up to 256K. Seven state-of-the-art algorithms, namely PT-Tree [40], TupleTree [39], DynamicTuple [34], HybridTSS [37], NuevoMatch [36], TupleMerge [33] and CutSplit [30], are selected as baselines. The experiment results demonstrate that DBTable outperforms these seven algorithms in terms of lookup performance, while also achieving the fastest update speed. Specifically, DBTable achieves an overall lookup speed improvement ranging from 1.53x to 7.29x for rulesets of size 256k.

The main contributions of this paper are as follows:

- We propose an efficient index structure for partitioning rules, eliminating rule replication by leveraging the strengths of traditional and learned indexes.
- We devise an iterative method for extracting a DBS from the ruleset, serving as a crucial foundation for achieving superior lookup performance.
- We introduce a skewness-aware hybrid indexing scheme, utilizing auxiliary TSS structures to improve worst-case lookup performance.

The remainder of this paper is organized as follows. Section II presents the background information. In Section III, we review and summarize the related literature. The design of the proposed DBTable index is elaborated in Section IV. Section V offers a detailed analysis of the experiment results. Section VI concludes the paper.

II. PRELIMINARIES

The objective of network packet classification is to achieve differentiated packet processing through the utilization of a classifier or ruleset, which encompasses a collection of predefined rules. Each rule, denoted as $R = \{r_1, r_2, \dots, r_F\}$, comprises F components, where r_i defines a condition on a field f_i . Moreover, each rule is assigned a priority and an associated action. The common five fields include source IP address (SIP), destination IP address (DIP), source port (SP), destination port (DP), and protocol (PRO). Typically, conditions defined on fields can be categorized into three types: prefix for IP addresses, exact or wildcard(*) for protocols, and range for port numbers.

Similarly, each packet, denoted as P , comprises F fields. Each field, with a size of d bits, has an associated value domain, usually ranging from 0 to $2^d - 1$. For a given packet $P = \{p_1, p_2, \dots, p_F\}$, p_i represents the value associated with field f_i . Thus, the process of packet classification entails identifying the rule within the classifier that exhibits the highest priority and matches the packet, subsequently associating the appropriate action corresponding to the matched rule.

III. RELATED WORK

In this section, we briefly introduce the existing algorithmic methods from four categories and provide a summary.

A. Decision Tree-Based Methods

The fundamental concept underlying decision tree-based methods is to recursively partition the value domain space of fields into smaller subspaces until the number of rules contained within each subspace falls below a specified threshold. This recursive splitting process allows for efficient lookup performance in most decision tree-based schemes, as they typically only traverse a single leaf node during the lookup process. However, an inherent challenge faced by these methods is the issue of rule replication, as a single rule may span multiple subspaces. Consequently, these methods do not effectively support rule updates.

HiCuts [19] is a classical scheme that introduces a cutting method to divide the search space into equal-sized subspaces. Building upon HiCuts, HyperCuts [20] enhances tree heights by incorporating a flexible cutting scheme, while HyperSplit [21] proposes a splitting method to address the rule replication problem. To achieve high performance while mitigating rule replication, some approaches adopt a strategy of constructing multiple trees. EffiCuts [22] utilizes a variant of HyperCuts to independently construct multiple trees. In contrast to the aforementioned methods, PartitionSort [26] introduces a multi-key binary search tree that effectively eliminates rule replication.

BitCuts [27] and ByteCuts [28] are two decision tree schemes that leverage fine-grained features to partition rules effectively. BitCuts introduces a “bit-level cut” method to minimize unnecessary partitions and mitigate excessive memory consumption. However, performing cuts within the space formed by a limited number of bits can lead to increased overlap among partitions, resulting in a higher incidence of rule replications. To address this problem, ByteCuts employs a strategy of constructing multiple trees and grouping rules with similar characteristics within the same tree. In contrast, CutSplit [30] employs a hybrid approach that combines cutting and splitting techniques. While CutSplit significantly reduces pre-processing time, the cost associated with tree reconstruction for updates remains prohibitively high for incremental updates.

B. TSS-Based Methods

Different from the decision tree-based methods, Tuple Space Search (TSS) [31] was the pioneering approach to construct a tuple space based on the prefix length combination of different fields. Specifically, rules are initially partitioned into distinct tuples and subsequently stored in a hash table using the prefixes as keys, resulting in efficient rule updating. Nevertheless, TSS necessitates an exhaustive search within the tuple space, leading to suboptimal lookup performance. To mitigate this issue, Priority Sorting Tuple Space Search (PSTSS) [32] introduces a priority-based sorting mechanism for tuples, enabling early termination during tuple probing. However, the worst-case scenario for PSTSS remains the same as that of TSS.

Recently, two approaches, namely TupleMerge (TM) [33] and DynamicTuple (DT) [34], have emerged to enhance lookup performance by adopting the concept of tuple merging. TM merges tuples by examining their similarity, whereas DT

utilizes dynamic programming to identify tuples that require merging. However, it should be noted that both methods may exacerbate hash collisions, which could potentially impact performance.

C. Neural Network-Based Methods

Neural network technology has emerged as a promising solution for diverse domains, including packet classification. One notable contribution in this regard is *NeuroCuts* [35], which leverages deep reinforcement learning (RL) to enhance cutting or splitting based on decision trees, thus yielding exceptional lookup performance. Similarly, *HybridTSS* [37] also harnesses RL to recursively construct multi-level tuples, thereby improving lookup performance. Despite the superiority of RL-based methods over heuristic approaches in terms of performance, it is important to note that they are unable to completely address the limitations inherent in the underlying base methods.

In contrast to the aforementioned approaches, *NuevoMatch* (NM) [36] presents a novel solution, namely the Range Query Recursive Model Index (RQ-RMI), which is built upon learned indexes. This technique effectively compresses a majority of the rules into model weights and replaces packet classification with model inference computation. By embedding the compressed model in the hardware cache, NM achieves a substantial improvement in lookup speed. However, due to the challenge of guaranteeing model prediction accuracy, the applicability of this method is limited.

D. Hybrid Methods

Some hybrid methods have been proposed to achieve enhanced lookup and update performance. *CutTSS* (CT) [38] and *TupleTree* (TT) [39] are two notable approaches that combine decision tree-based and TSS-based schemes. CT leverages a TSS-based approach for large nodes after constructing decision trees. By ensuring the absence of rule replication, CT achieves superior update performance compared to other decision tree-based methods. In contrast, TT employs alternating partition and hash strategies to recursively construct a decision tree. Through the introduction of a partition method based on the maximum common tuple, TT surpasses CT in terms of lookup performance. Additionally, *PT-Tree* (PT) [40] proposes a hash tree structure and maps rules into leaf nodes layer by layer. By utilizing fine-grained bytes in rules to construct hash tables, PT achieves a concise and efficient structure. However, these methods involve compromises in lookup performance, requiring multiple leaf nodes to be accessed during packet classification.

E. Summary

Table I presents a comparison of the structure and underlying principles of recent algorithmic methods. It is evident that achieving high performance in both lookup and update operations poses a challenge for these methods. To address this challenge, we propose a novel scheme called *DBTable*, which exclusively constructs a hash table based on a compact DBS.

TABLE I
COMPARISON OF RECENT ALGORITHMIC SCHEMES

Category	Scheme(year)	Data Structure	Lookup Method ¹	Update Method ²
Decision tree-based	BitCuts(2017) [27]	Tree	TSL	N
	ByteCuts(2018) [28]	Tree	TML	N
	CutSplit(2018) [30]	Tree	TML	N
Tuple space-based	TupleMerge(2019) [33]	Hash Table	TTS	MT
	DynamicTuple(2022) [34]	Hash Table	TTS	ST
Neural network-based	NeuroCuts(2019) [35]	Tree	TSL/TML	N
	NuevoMatch(2020) [36]	Neural Network	MI	N
	HybridTSS(2022) [37]	Multi-level Hash	TTS	MT
Hybrid scheme	CutTSS(2020) [38]	Tree & Hash	TML	MN
	TupleTree(2022) [39]	Tree & Hash	TML	MN
	PT-Tree(2023) [40]	Hash Tree	TML	SL
	DBTable	Hash Table	AST	ST

¹**Lookup Method:** TSL and TML represent the traversing of a single leaf and multiple leaves, respectively. TTS denotes the traversing of the tuple space. MI and AST stand for model inference and accessing a single table, respectively.

²**Update Method:** N indicates the lack of support for incremental updating. ST and MT represent the modification of a single table and multiple tables, respectively. SL and MN denote the modification of a single leaf and multiple nodes, respectively.

While the design of *DBTable* draws inspiration from existing methods, notable disparities exist in terms of conceptualization and algorithmic approach when compared to these methods.

Firstly, the hybrid structure of *DBTable* is specifically designed to alleviate worst-case scenarios when compared with existing hybrid structure methods [38], [39], [40]. The cornerstone of *DBTable* lies in the hash table constructed based on DBS. In contrast, existing hybrid methods often adopt strategies to partition the ruleset for different methods to fully harness the advantages of each approach. In comparison, *DBTable* relies less on synthetic methods and is dedicated to minimizing the construction of auxiliary structures from the beginning, as will be further elucidated in the experimental section. The streamlined and efficient structure rooted in DBS enables *DBTable* to surpass its counterparts.

Secondly, concerning the feature extraction method, *DBTable* is dedicated to transforming multi-dimensional data into one-dimensional data while preserving separable characteristics. This approach establishes the foundational requirement for constructing streamlined and efficient data structures. In contrast, existing methods [27], [28], [40] typically adopt segment-based approaches, gradually separating rules by combining multi-dimensional features extracted from segments. Although segment-based feature extraction enhances the separability of local rule subsets, the data structure requires more information to be stored. Furthermore, multi-segment structures need to be accessed during querying, thereby impacting efficiency.

Finally, *DBTable* integrates concepts from various machine learning methods and addresses their shortcomings. Neural network methods are typically utilized to optimize the structure of existing methods by learning features that are challenging to intuitively discern within the architecture, with optimization results constrained by the inherent characteristics of the original structure. Different from these, *DBTable* systematically examines the characteristics of the ruleset before designing the data structure. Additionally, the minimalist structure design and partition method of *DBTable* simplify the feature extraction problem, allowing for the intuitive formu-

lation of feature extraction metrics. Compared to machine learning methods, DBTable is not only easier to interpret, but also avoids issues like erroneous predictions seen in methods such as NuevoMatch [36].

IV. ALGORITHM DESIGN

A. Basic Idea

To optimize lookup performance, it is possible to achieve $O(1)$ complexity in memory access by building a hash table for all points in the searching space. However, to achieve this, the complexity of storage is $O(2^m)$, where m represents the total number of bits occupied by all fields. On the other hand, optimizing storage to achieve $O(n)$ complexity will increase lookup complexity up to $O(n \log^{F-1})$, where n and F represent the number of rules and fields respectively [43]. Obviously, both of these extremes are impractical because packet classification has high performance requirements for both lookup time and memory footprint.

Machine learning models [35], [36] have been used to optimize or replace traditional data structures for packet classification. In particular, NM [36] turns rule matching into a model inference by applying an RMI method [44]. Therefore, NM can directly locate the possible matching rules through model inference, and the concise model can be accommodated in a CPU high-level cache. Although learned indexes can effectively extract features from rulesets to model the relationship between packets and matching rules, it requires complex inference processes and a long model training time, which affects lookup and update performance.

We aim to combine the advantages of traditional and learned indexes to achieve fast lookup and update performance. Inspired by learned indexes, we believe that there are discriminative features in rulesets that can be extracted to partition rules. In fact, each rule can be regarded as a set of bits and each bit represents a dimension. Therefore, the problem of feature extraction is equivalent to the problem of dimensionality reduction [45], namely extracting a discriminative bitset (DBS) that can effectively classify rules. Building indexes based on DBS has two benefits. First, the index structure can achieve nearly $O(1)$ lookup and update performance. Second, the index space is also reduced, ensuring a low memory footprint. Therefore, we propose DBTable, whose core concept is to index rules on DBS.

B. Indexing Scheme

Since some unspecified bits exist in rules, namely wildcard bits denoted by *, we need an effective way to handle them in DBS when indexing rules. To solve this problem, it is a common practice to make two rule replicas for each wildcard bit, such as ByteCuts [28] and BitCuts [27]. The advantage of this method is that only one index path needs to be accessed during lookup due to rule replication. However, if the DBS of a rule contains l wildcard bits, it will be replicated 2^l times, which not only incurs a high memory overhead, but also makes rule updates inefficient.

To avoid rule duplication while ensuring lookup efficiency, we propose a concise indexing scheme based on DBS. Initially,

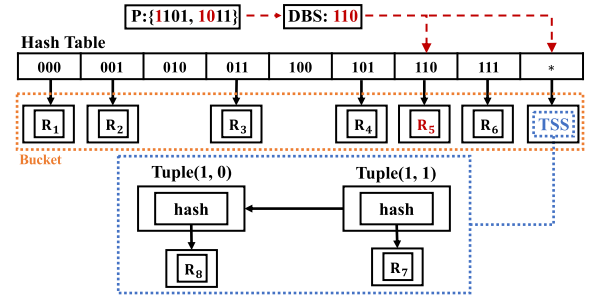


Fig. 1. The structure of DBTable constructed based on a DBS with 3 bits, where a TSS structure is built in bucket * according to the rules stored in it.

we divide the rules into two categories: wildcard rules C_w and non-wildcard rules $C_{\bar{w}}$. Specifically, a rule belongs to C_w if its DBS contains at least one wildcard. All rules that do not belong to C_w belong to $C_{\bar{w}}$. Then, for non-wildcard rules, the DBS is interpreted as a field on which we build a hash table. Since the DBS has good separability, these rules can be separated into buckets evenly. Note that all rules belonging to C_w are mapped to a special wildcard bucket denoted as *.

Figure 1 shows the index structure of DBTable which builds a hash table based on the DBS. Each element in the table points to a bucket that stores rules. For l bits in the DBS, the length of the hash table is $2^l + 1$. The first 2^l buckets store rules belonging to $C_{\bar{w}}$, while all rules belonging to C_w are stored in the wildcard bucket *. DBTable can effectively avoid rule replication, ensuring update and lookup performance. In addition, at most two buckets need to be accessed during lookup. One is determined by the DBS of the packet, and the other is the wildcard bucket.

C. An Iterative Method for Extracting DBS

1) *Overview*: The extraction of DBS is the key issue of DBTable, which significantly affects the overall performance. Therefore, when extracting DBS, we consider two main objectives. First, the DBS should separate the rules belonging to $C_{\bar{w}}$ evenly, ensuring the overall lookup performance. Second, the DBS should minimize the number of rules belonging to C_w to be as low as possible, guaranteeing the worst-case lookup performance. When reaching these two objectives, the DBTable built on extracted DBS can achieve a high lookup performance while avoiding excessive memory consumption.

2) *Algorithm Design*: As described in Section IV-C.1, the primary goal of DBS extraction is manifested in two aspects: evenly partitioning the ruleset into smaller subsets and avoiding an excessive number of rules belonging to C_w in the partitions. Firstly, the characteristics of chosen bits determine their suitability for evenly partitioning a ruleset. Therefore, a metric needs to be developed to measure the separability of optional bits. Secondly, as there are correlations between different bits, the method of combining bits will also affect the final separation effect. Thus, a criterion for selecting bits must be proposed to ensure the efficacy of the chosen bits. Finally, there is a clear tradeoff between the number of selected bits and the extent of rule separation. However, selecting a large number of bits will significantly increase storage overhead. Therefore, it is essential to establish a suitable termination

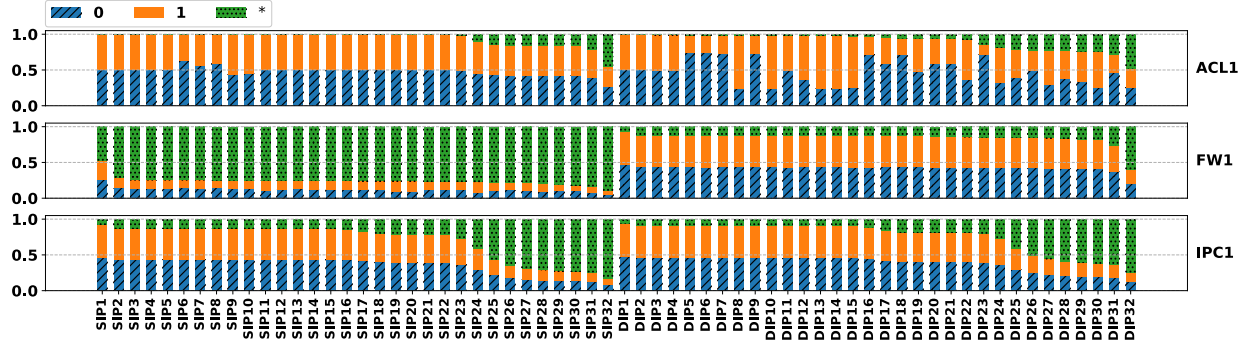


Fig. 2. The distribution of bits in three rulesets.

condition for extraction to achieve efficient rule partitioning with fewer bits. Based on the analysis above, we design a method for DBS extraction from three key perspectives: i) How to quantify the separability of bits? ii) What are the criteria for selecting bits? and iii) What are the conditions for terminating extraction?

a) *Quantification of bit separability*: Figure 2 shows the distribution of 64 bits of SIP and DIP fields in three 5-field rulesets, namely ACL1 (Access Control List), FW1 (Fire Wall) and IPC1 (IP Chain). The blue, orange and green bars represent the proportion of rules that specify 0, 1 and * respectively on each bit. Although the distribution is not the same in the three rulesets, there are some bits that have an equal distribution between 0 and 1, and the proportion of * is very small, such as SIP1 to SIP5 in ACL1. By indexing based on any of these bits, we can evenly partition rules belonging to C_w into two subsets, and the subset corresponding to * is very small, meeting the two objectives.

Based on this observation, to quantify the discrimination of each bit b_i in a ruleset, we define a metric as:

$$\mu_i = |\gamma_i^0 - \gamma_i^1| + \gamma_i^* \quad (1)$$

where γ_i^0 , γ_i^1 and γ_i^* represent the proportion of rules that specify 0, 1 and * on b_i , respectively. The $|\gamma_i^0 - \gamma_i^1|$ term measures the uniformity of the distribution of 0 and 1 on b_i , corresponding to the first objective. The more uniform, the smaller the value. The γ_i^* term measures the number of rules belonging to C_w , corresponding to the second objective. As a result, μ_i comprehensively considers the two objectives for extracting DBS.

The range of μ is $[0, 1]$, and only when $\gamma^0 = \gamma^1$ and $\gamma^* = 0$, $\mu = 0$, which means that the bit can separate the ruleset into two equal-size subsets. It is better for b_i to have smaller $|\gamma_i^0 - \gamma_i^1|$ and γ_i^* . Smaller μ_i indicates that b_i has better separability and is more likely to be selected. According to our analysis, when μ is greater than 0.5 for a bit, it is difficult for the bit to separate the ruleset evenly, so we do not select bits with μ greater than 0.5.

b) *Selection method of DBS*: Based on μ , the next question is how to select bits to form the DBS. A straightforward method is to select the k bits corresponding to the top- k smallest μ in a one-time manner. However, even though each bit has high discrimination, their combination may not be able to effectively separate the ruleset due to the possible correlation between different bits. To solve this problem,

we propose an iterative extraction method to avoid selecting highly correlated bits. When some bits have been selected to partition the ruleset into multiple subsets, since the distribution of bits in each subset may be changed, our proposed method recalculates μ on each subset and forms the DBS by iteratively selecting bits.

Specifically, given a group of subsets $\mathbb{S} = \{\mathbf{S}_i, i \in \mathbb{N}_+\}$, for each subset $\mathbf{S}_i$, we first select the bits that correspond to the top- k smallest μ as a set of candidate bits $\mathbf{B}_i$. In this way, we obtain a group of bitsets $\mathbb{B} = \{\mathbf{B}_i, i \in \mathbb{N}_+\}$. Note that we do not select the bit with μ exceeding 0.5. Then, we select a common bit from $\mathbb{B}$ which should have as much good separability for most subsets as possible. Based on the bit, each current subset is partitioned into at most three subsets to form a new group of subsets $\mathbb{S}_{new}$. This process is repeated and a bit is selected in each iteration.

Obviously, the selection of the common bit in each iteration is important, which should be considered from a global perspective. First, the bit should have a large influence range (IR) that can partition as many subsets as possible. Second, the bit should have good overall separability (OS), meaning its μ should be as small as possible in most subsets. These two requirements ensure that the selected bit in each iteration is effective for separating the ruleset.

Based on the above analysis, we define a metric θ which can be used as the basis for selecting the common bit in each iteration. Given $\mathbb{S}$, for each bit b_i in $\mathbb{B}$, we define θ_i as a pair of IR and OS:

$$\theta_i = \left\langle \sum_{j \in \{x | b_i \in \mathbf{B}_x\}} |\mathbf{S}_j|, \sum_{j \in \{x | b_i \in \mathbf{B}_x\}} \mu_i^j \right\rangle \quad (2)$$

The first item represents the IR of b_i , which accumulates the number of rules in all subsets that select b_i as a candidate bit. The larger the IR, the more rules b_i can separate. The second item corresponds to OS, which sums the μ_i in all subsets that select b_i . The smaller the value, the better the overall separability. In each iteration, we prioritize the selection of the bit with the largest IR. When there are multiple bits with the same IR, we select the bit with the smallest overall μ_i .

c) *Termination conditions*: Since we iteratively extract bits, when to stop is another key issue. Balancing lookup performance and memory footprint, we set two conditions to terminate the iteration: i) whether the newly selected bit can further partition subsets. If the bit can no longer continue to separate at least one new subset, the iteration should

TABLE II
EXAMPLE RULES

Rule ID	Field X	Field Y	Tuple	Extracted DBS
R_1	0101	0010	(4, 4)	000
R_2	0101	0110	(4, 4)	001
R_3	0001	111*	(4, 3)	011
R_4	110*	010*	(3, 3)	101
R_5	110*	10**	(3, 2)	110
R_6	11**	11**	(2, 2)	111
R_7	1***	0***	(1, 1)	10*
R_8	0***	****	(1, 0)	0**

TABLE III
THE CALCULATION RESULTS OF μ IN EACH SUBSET

	$S_{1,1}$	$S_{2,1}$	$S_{2,2}$	$S_{3,1}$	$S_{3,2}$	$S_{3,3}$	$S_{3,4}$	$S_{3,5}$
μ_1	0	-	-	-	-	-	-	-
μ_2	0.75	0.5	1	1	1	1	1	1
μ_3	1	1	1	1	1	1	1	1
μ_4	1	1	1	1	1	1	1	1
μ_5	0.25	0.5	0	-	-	-	-	-
μ_6	0.5	0.5	0.5	0	1	1	1	0
μ_7	0.75	1	1	1	1	1	1	1
μ_8	1	1	1	1	1	1	1	1

be stopped; ii) whether the separated subsets meet performance requirements. This is used to limit memory overhead while maintaining lookup performance. Due to the exponential increase in memory overhead when adding a bit, the iteration should stop when most of the partitioned subsets are small. Therefore, we set a threshold $binth$ on the size of subsets. When the number of rules in a subset is less than $binth$, the subset meets the expected criteria. In addition, we set a threshold th_b on the proportion of the expected subsets. The iteration should stop when the proportion exceeds th_b .

3) *Example*: We provide an example to illustrate the DBS extraction process. Table II lists 8 example rules. For convenience, we use two fields X and Y , each of which occupies 4 bits. We label the 8 bits as b_1 to b_8 in order from the highest bit of field X to the lowest bit of field Y . We set $binth$, th_b and k to 1, 0.8 and 2, respectively. Table III and Table IV present the μ of bits calculated in each subset and the θ of bits at each iteration. In the two tables, we mark the selected candidate bits in each subset and the selected common bit at each iteration in red.

Figure 3 shows the bits selected at each iteration and how subsets are partitioned iteratively. At the first iteration, we calculate μ for each bit in $S_{1,1}$ and select b_1 and b_5 as candidate bits. For example, $\mu_1 = |0.5 - 0.5| + 0 = 0$, since 4 rules specify 0 and the other 4 rules specify 1 on b_1 in $S_{1,1}$. Then, we obtain θ_1 and θ_5 based on μ_1 and μ_5 . Since there is only one subset, the IR of both θ_1 and θ_5 is 8. Therefore, we compare the OS of θ_1 and θ_5 and select the bit corresponding to the smaller one, namely b_1 . $S_{1,1}$ is partitioned into two subsets based on b_1 , namely $S_{2,1}$ and $S_{2,2}$. The second iteration is the same. At the third iteration, since the size of all subsets is less than $binth$, the extraction process is stopped. The final DBS contains b_1 , b_5 and b_6 . The DBS of each rule is also shown in Table II.

4) *DBS Extraction Algorithm*: The procedure of iteratively extracting DBS is shown in Algorithm 1. Before iteration, we first initialize the extracted DBS B_e and add ruleset R

TABLE IV
THE CALCULATION RESULTS OF θ AT EACH ITERATION

Iteration	θ_1	θ_2	θ_3	θ_4	θ_5	θ_6	θ_7	θ_8
1	(8, 0)	-	-	-	(8, 0.25)	-	-	-
2	-	-	-	-	(4, 0)	-	-	-
3	-	-	-	-	-	(4, 0)	-	-

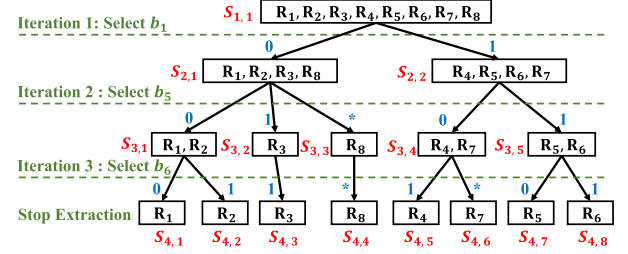


Fig. 3. The example of DBS extraction.

to the group S (line 1). When the set of optional bits B_o is not empty, the extraction process iterates. Each iteration can be divided into three parts: 1) calculating μ and θ , and selecting a bit; 2) separating the current subsets based on the selected bit; and 3) checking the termination conditions.

In the first part, we initialize an array A_θ to store θ and then calculate μ of each bit in B_o for each subset S (lines 3-7). Then, we update A_θ according to the top- k smallest μ in each subset S (lines 8-10). The update process adds $|S|$ and μ to the first and second items of θ corresponding to the subscript $\mu.idx$ in A_θ , respectively. After processing all subsets, we select a common bit b_s according to A_θ (line 11).

In the second part, we partition each current subset based on b_s , forming a new group of subsets S_{new} (lines 12-14). The partition method is simple. According to the value specified on b_s , each rule is partitioned into one of the three subsets corresponding to 0, 1 and *.

In the third part, we judge whether to stop iterating. If no new subset is obtained, meaning b_s has no separation, we stop the iteration and return B_e (lines 15-16). Otherwise, b_s is added to B_e and removed from B_o (line 17). Then, we count the number of expected subsets whose size is less than $binth$. If the proportion of the expected subsets to the total ones exceeds th_b , the iteration is stopped (lines 18-22).

The complexity of Algorithm 1 is $O(mn + n \log n)$, where n and m represent the total number of rules and optional bits, respectively. At each iteration, the algorithm contains three parts. For the first part, the algorithm traverses all rules, therefore the complexity is $O(n)$. The second part is mainly to find the top- k smallest μ in each array A_μ which stores the calculated μ and finds the best θ in A_θ . Both procedures can be implemented with $O(m)$ complexity. In the third part, the algorithm also traverses all rules and separates them into new subsets, taking the complexity of $O(n)$. The algorithm needs to iterate up to $\log n$ times, assuming that each iteration evenly partitions the subsets. Therefore, the total complexity is $O(mn + n \log n)$.

Algorithm 1 should extract at least $\log n$ bits to fully separate n rules. Consequently, if each bit is considered a dimension, Algorithm 1 can limit the dimensions to approximately $\log n$, irrespective of the initial dimension m . Given

Algorithm 1 Iterative DBS Extraction

Input: ruleset $\mathbf{R}$, optional bitset $\mathbf{B}_o$, binth , th_b and k
Output: the extracted DBS $\mathbf{B}_e$

```

1  $\mathbf{B}_e \leftarrow \emptyset$ , add  $\mathbf{R}$  to  $\mathbf{S}$ ;
2 while  $|\mathbf{B}_o| \neq 0$  do
3   Initialize an array  $\mathbf{A}_\theta$ ;
4   foreach subset  $S$  in  $\mathbf{S}$  do
5     Initialize an array  $\mathbf{A}_\mu$ ;
6     foreach bit  $b$  in  $\mathbf{B}_o$  do
7       calculate  $\mathbf{A}_\mu[b.\text{id}x]$ ;
8     foreach top- $k$  smallest  $\mu$  in  $\mathbf{A}_\mu$  do
9       if  $\mu < 0.5$  then
10        update  $\mathbf{A}_\theta[\mu.\text{id}x]$ ;
11   $b_s \leftarrow$  select the common bit according to  $\mathbf{A}_\theta$ ;
12   $\mathbf{S}_{\text{new}} \leftarrow \emptyset$ ;
13  foreach subset  $S$  in  $\mathbf{S}$  do
14    partition  $S$  by  $b_s$  and add partitioned subsets into  $\mathbf{S}_{\text{new}}$ ;
15  if  $|\mathbf{S}| == |\mathbf{S}_{\text{new}}|$  then
16    return  $\mathbf{B}_e$ ;
17  add  $b_s$  to  $\mathbf{B}_e$ , remove  $b_s$  from  $\mathbf{B}_o$ ;
18   $\mathbf{S} \leftarrow \mathbf{S}_{\text{new}}$ ,  $ES \leftarrow 0$ ;
19  foreach subset  $S$  in  $\mathbf{S}$  that  $|S| < \text{binth}$  do
20     $ES \leftarrow ES + 1$ ;
21  if  $ES/|\mathbf{S}| > \text{th}_b$  then
22    return  $\mathbf{B}_e$ ;

```

that the number of rules n is usually significantly smaller than the total space size 2^m formed by m bits, it can be deduced that $\log n < m$. For instance, in the classic IPv4 protocol, the two IP fields, SIP and DIP, comprise 64 bits, corresponding to 64 dimensions. For a ruleset of size 256k, Algorithm 1 can reduce the dimensions to 18, thereby significantly decreasing the hashing complexity. In comparison to the TSS method, this reduction allows the hashing space to be reduced from 2^{65} to 2^{18} , assuming TSS utilizes SIP and DIP to construct tuples. Therefore, DBTable can achieve high overall performance while preserving a compact structure.

D. Skewness-Aware Hybrid Structure

Although the DBS can effectively distribute most rules evenly, a few buckets that harbor a disproportionate number of rules may exist, consequently impacting the worst-case lookup performance. This issue primarily arises due to two factors. First, the presence of data skew within the ruleset makes it challenging to evenly distribute certain rules. Second, DBTable considers the storage overhead, leading to the aggregation of certain separable rules within a single bucket.

To solve this problem, we employ a hybrid structure to ensure the worst-case lookup performance. Specifically, when the size of a bucket exceeds a threshold th_c , we construct a TSS structure [34] for the bucket. Using TSS as the second-level index structure can have two advantages. First,

TSS supports efficient rule updates. Second, after being partitioned by DBS, the rules in each bucket have high similarity. This means that fewer tuples need to be constructed using TSS, thereby achieving high lookup and storage efficiency.

E. Operations on DBTable

1) *Construction*: Based on the hybrid structure, the construction of DBTable can be divided into two parts: constructing buckets and adjusting buckets. First, we construct $2^l + 1$ buckets according to the bit number l of DBS, and directly insert rules into the corresponding buckets based on the indexing scheme. Then, we traverse all buckets. When the size of the bucket exceeds the threshold th_c , we build a TSS structure to replace the bucket. DBTable mainly consists of hash tables. When the number of rules is n , it requires storage space with a worst-case complexity of $O(n)$.

Figure 1 shows the structure built with the rules in Table II, where th_c is set to 1. First, according to the extracted DBS shown in Table II, $R_1 - R_6$ are inserted into the corresponding buckets based on the value of the DBS. Since the DBS of R_7 and R_8 contain wildcard bits *, they are inserted into the wildcard bucket *. Since the size of bucket * is greater than th_c , we construct a TSS structure in which R_7 and R_8 are inserted into the corresponding tuple, as shown in Figure 1.

2) *Lookup*: The lookup process of DBTable is very straightforward. Given a packet, at most two buckets need to be accessed. One bucket corresponds to the value of the DBS of the packet and the other is the wildcard bucket *. If the bucket directly stores rules, we check them one by one to search matching rules. Otherwise, we continue to access the TSS structure. Therefore, assuming DBTable constructs up to t tuples in each bucket, the lookup time complexity of DBTable is $O(t)$.

For example, in Figure 1, the DBS value of packet P : $\{1101, 1011\}$ is $\{110\}$, expressed in binary format. Therefore, only two buckets 110 and * need to be accessed (as depicted by the dotted arrow lines), ultimately acquiring the matching rule R_5 . We also take advantage of the rule priority information to achieve early termination. By recording the highest priority of the currently obtained matching rules, we can quickly filter out the buckets that store rules with lower priority. Since DBTable can separate most rules in the first-level index, usually one memory access is required in the lookup process, which ensures high lookup performance.

3) *Update*: Rule updates can be divided into two situations. One is incremental rule updates, and the other is ruleset switching. In most cases, incremental updating only changes a few rules by modifying, adding, or removing them, without significantly altering the ruleset distribution. DBTable shows efficient performance in updating as inserting or deleting a rule only affects one bucket. In the worst scenario, locating a rule may involve accessing multiple tuples in the substructure. However, typically, DBTable only needs to access a single bucket corresponding to the rule's DBS. Due to the limited number of rules in a bucket, the overall insertion and deletion speeds maintain high even if some rules need to be updated.

For ruleset switching, the previously extracted DBS may no longer be suitable for partitioning the ruleset. To ensure lookup

performance, we can re-extract DBS and rebuild the structure. Fortunately, DBTable only needs to extract DBS once during the construction process, so its construction time is comparable with that of some state-of-the-art algorithms, as presented in Section V.

F. Dynamic Reconfiguration

The core of DBTable is primarily data-driven. Therefore, to ensure lookup performance, DBTable needs to be reconfigured when there are significant changes in the ruleset. However, identifying the optimal timing for reconfiguration poses a significant challenge. One straightforward approach is to set a traffic window and re-extract the DBS based on the current ruleset within each window. The number of differing bits between the currently used DBS and the newly extracted DBS is then compared. If the difference exceeds a predefined threshold th_s , a new DBTable index structure is constructed based on the new DBS, replacing the original structure. This method can ensure that DBTable maintains high lookup performance during ruleset transitions.

However, performing DBS re-extraction in each window incurs considerable additional computational overhead. Furthermore, relying solely on the number of differing bits to reconstruct the index structure may lead to inefficient resource utilization. Although there may be disparities between the current DBS and the new DBS, the impact on lookup performance might be minimal in certain scenarios. Therefore, we introduce a monitor of lookup performance to determine when to re-extract the DBS.

Specifically, we set a lookup time growth threshold th_L . When the growth ratio exceeds th_L , DBS re-extraction is triggered. Initially, we take the average lookup time of the first window after the initial index construction as the baseline, denoted as L_{base} . Then, we monitor the average lookup time of each subsequent window. If the average lookup time of the current window, L_{cur} , increases by more than th_L compared to L_{base} ($L_{cur}/L_{base} - 1 > th_L$), we re-extract the DBS based on current ruleset. Subsequently, we compare the number of differing bits and decide whether to rebuild the index based on th_s . When an index reconstruction takes place within a window, we promptly update L_{base} . By employing this dual-check approach, we can effectively decrease the frequency of DBS extraction and index reconstruction while ensuring lookup performance stability.

G. Scalability

As networks become increasingly complex, higher requirements are placed on the scalability of packet classification algorithms. DBTable has good scalability in terms of both the number of fields and the type of fields. First of all, DBTable can cope well with changes in the number of fields. Although the increase in the number of fields will lead to a large set of optional bits, when the size of the ruleset does not change, the number of extracted bits remains almost the same. In addition, more optional bits mean that there may be more discriminative bits, which is beneficial to DBTable.

For different types of fields, DBTable can directly support fields in the form of prefixes and single values, such as IP address, MAC address and protocol. However, when dealing with fields represented by ranges in their conditions, such as port, DBTable cannot directly extract bits from them. Therefore, we propose a conversion method that enables DBTable to support such fields when extracting DBS, thereby improving the adaptability of DBTable.

Specifically, a range constraint r_i on a field f_i with a size of d bits can be represented as $[\{b_1^{low} \dots b_d^{low}\}, \{b_1^{high} \dots b_d^{high}\}]$, where $\{b_1^{low} \dots b_d^{low}\}$ denotes the binary representation of the lower bound of the range, and $\{b_1^{high} \dots b_d^{high}\}$ represents the upper bound. Assuming the longest sequence of identical bits starting from b_1 between the lower and upper bounds is j , then the first j bits of p_i in packet P must match $\{b_1^{low} \dots b_j^{low}\}$ or $\{b_1^{high} \dots b_j^{high}\}$ for p_i to potentially match r_i . For the remaining bits, any value assigned to p_i may match r_i .

Therefore, we can convert a range constraint r_i into a prefix representation $\{b_1 \dots b_d\}$ for DBS extraction, where $\{b_1 \dots b_j\}$ are the same as $\{b_1^{low} \dots b_j^{low}\}$ and $\{b_1^{high} \dots b_j^{high}\}$, and $\{b_{j+1} \dots b_d\}$ are specified as wildcard $*$. For example, the range $[1, 3]$ defined on a 4-bit field can be converted into the prefix $\{00**\}$. By adopting this conversion approach, DBTable can handle fields represented by ranges and prefixes in a unified way during DBS extraction.

V. EXPERIMENTS

A. Experiment Setup

We conduct experiments to comprehensively evaluate the performance of DBTable (DBT) from four aspects: lookup time, update time, construction time, and memory footprint. We compare DBT with seven state-of-the-art algorithms, namely PT-Tree (PT) [40], TupleTree (TT) [39], Dynamic-Tuple (DT) [34], HybridTSS (HT) [37], NuevoMatch (NM) [36], TupleMerge (TM) [33], and CutSplit (CS) [30]. These algorithms are described in detail in Section III. We use the open-source code of these baseline algorithms. We also open the source code of DBT.¹

The default experimental environment uses a physical machine equipped with an AMD EPYC-7542 32-core CPU@2.9GHz, 252GB of DRAM, and the Ubuntu 18.04.6 operating system. The tested algorithms are compiled with GCC 7.5.0 with O3 optimization in the default environment. The parameters of each algorithm are set according to the recommended settings provided in the respective work. NM employs TM as the external classifier and uses 1 iSet with a batch size of 128 during lookup. The collide parameter of TM is set to 10. For CS, $binth$ is set to 8 and the threshold for the prefix length of *big field* is set to 20.

To evaluate the practicability of DBT in real-world applications, we constructed a DPDK² environment on a virtual machine with 8 cores and 32GB DRAM. The DPDK, an open-source Data Plane Development Kit, consists of libraries that

¹<https://github.com/lavline/DBTable.git>

²<https://www.dpdk.org>

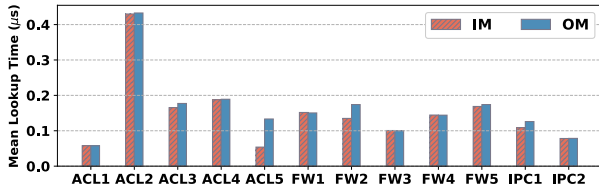


Fig. 4. Comparison of two methods for extracting DBS.

accelerate packet workload processing and is widely used in real-world scenarios [46], [47]. The DPDK provides an Access Control library that enables the classification of input packets based on a set of rules. Therefore, we compared DBT with the packet classifier implementation `rte_acl` in the DPDK Access Control library. We use the DPDK testing tool to evaluate their performance. The operating system is Ubuntu 24.04 with GCC 13.2.0, and all code is built with O3 optimization. The DPDK version is 24.03.0, binding 2 VMXNET3 paravirtualized NICs. Each NIC sets 1 receive queue and 1 transmit queue, each configured with 1 flow. The `rte_acl` API utilizes the scalar method.

The rulesets are generated by ClassBench-ng [42] based on 12 seeds, namely ACL1-ACL5, FW1-FW5 and IPC1-IPC2. Similar to existing work, we generate two scale rulesets with sizes of 128k and 256k. For the default packet sets, we develop a generator to randomly generate 1,000,000 matching packets based on each ruleset. Our generator produces packets without locality to evaluate the overall performance of algorithms. Additionally, we utilize ClassBench to generate packets with different levels of locality to evaluate the performance of different algorithms under skewed traffic. Each packet set contains 1,300,000 packets.

We also evaluated the lookup performance of different algorithms using a real packet trace obtained from WIDE.³ Since real traces are not related to ClassBench seeds, most packets do not align with any rules, resulting in only the *default* rule being applied [33]. Typically, such traffic is rejected. However, in practical scenarios, the volume of rejected traffic is usually lower than accepted traffic [48], with higher rejection rates primarily observed during network attacks [49]. Therefore, we mix randomly generated packets with real packets at varying proportions, ranging from 10% to 30%. Each mixed traffic set comprises 1,000,000 packets.

B. Ablation Studies

We compare the iterative method (IM) with the baseline of the one-time method (OM) for DBS extraction. According to the number of bits l extracted by IM, OM directly selects the same number of bits corresponding to the first l smallest μ . Figure 4 shows the evaluation of these two methods. We can see that on 9 rulesets, the two methods have almost the same lookup time. This is because the correlation between bits with high separability in these rulesets is weak, so the DBS extracted using the two methods only has 1 or 2 different bits. However, when the correlation between bits is strong,

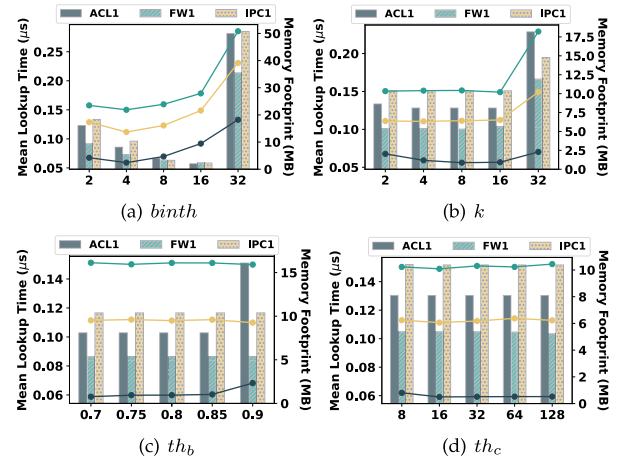


Fig. 5. Evaluation of parameter sensitivity, where lines and bars represent lookup time and memory footprint, respectively.

the separation effect of DBS extracted by OM decreases, thus affecting the lookup performance. For example, on ACL5, FW2 and IPC1, the lookup time of IM decreases significantly. Especially on ACL5, the lookup time of OM is 2.47 times that of IM. This verifies that IM is more effective than OM in avoiding the extraction of correlated bits to achieve better performance.

C. Sensitivity Studies

DBT has 4 parameters, namely $binth$, k , th_b and th_c . Of them, $binth$ and k are two key parameters for DBS extraction, while th_b and th_c are mainly used to guarantee the worst-case performance of DBT. We conduct experiments on ACL1, FW1 and IPC1 to evaluate the sensitivity of these four parameters. The default parameter settings are $binth = 4$, $k = 8$, $th_b = 0.8$ and $th_c = 16$. Since lookup performance is more critical in packet classification, we prioritize the impact on it, followed by memory footprint. Figure 5 shows the impact of each parameter when fixing the other three parameters on the three 256k rulesets, where lines and bars represent lookup time and memory footprint respectively.

1) $binth$: $binth$ significantly impacts the lookup performance and memory footprint of DBT. In general, as the value of $binth$ decreases, the number of rules to be accessed in each bucket decreases, resulting in faster checking within the buckets. On the other hand, a small $binth$ requires a larger DBS, resulting in an exponential increase in storage. As shown in Figure 5(a), when increasing $binth$, the lookup time shows an upward trend, while the memory footprint keeps decreasing. Note that a large $binth$, such as $binth = 32$, will result in many buckets having to build a TSS structure, thereby increasing the memory footprint. Similarly, the lookup time increases when $binth = 2$, since a too-small $binth$ leads to the extraction of low-discrimination bits, reducing the indexing efficiency of DBT. In practice, we use the size of cache blocks as a baseline to set $binth$. If a bucket can fit in the cache block, only one memory access is required to transfer the bucket. Therefore, to balance memory footprint and lookup speed, a larger $binth$ can be configured, as long as the storage required for $binth$ rules does not surpass the cache block

³<http://mawi.wide.ad.jp/mawi/samplepoint-F/2024/202401011400.html>

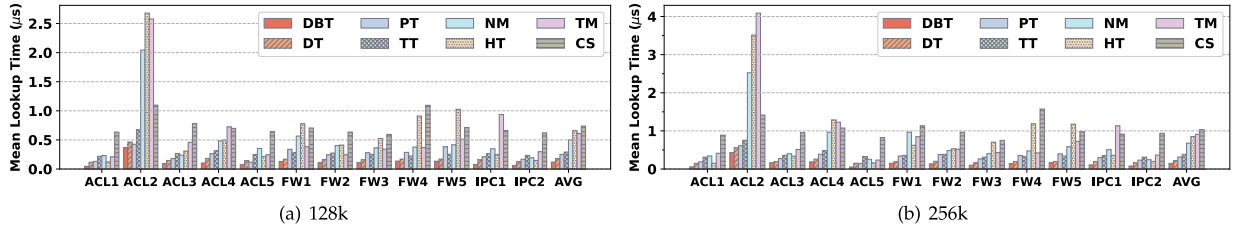


Fig. 6. Lookup time on non-local traffic.

size. In our experiments, $binth$ is set to 4 for the best lookup performance and a moderate memory footprint.

2) k : k determines the number of selected candidate bits in each subset when extracting DBS. For different rulesets, both a too-small and too-large k may lead to the selection of poorly discriminative bits, which reduces the lookup performance. For example, in Figure 5(b), the lookup time of ACL1 first decreases and then increases, and the lookup time of FW1 and IPC1 also increases significantly when $k = 32$. For the memory footprint, when $k = 32$, the discrimination of some extracted bits is too poor, therefore a larger size of DBS is required, which increases the storage significantly. For the best lookup performance, we set k to 8.

3) th_b : th_b is one of the termination conditions when extracting DBS. Since th_b is a lower bound guarantee for the degree of separation of rulesets, in most cases, the setting of th_b has little impact on performance when it is not too small, as shown in Figure 5(c). However, setting th_b too large may result in the additional selection of low-efficiency bits, increasing the memory footprint and lookup time, such as on ACL1 when $th_b = 0.9$. Based on the results, th_b is set to 0.8.

4) th_c : th_c is the threshold on bucket size to build the TSS structure, which ensures the worst-case lookup performance. Usually, th_c is set larger than $binth$ since the size of most buckets is less than $binth$. However, a too large th_c may deteriorate the worst-case performance. In our experiments, when th_c is tuned from 8 to 128 with a setting of $binth = 4$, the lookup time and memory footprint of the three rulesets are almost the same since few buckets have a size greater than 8, as shown in Figure 5(d). According to our observations on the other 9 rulesets, we set th_c to 16.

D. Performance Comparison

1) *Lookup Time*: In real-world scenarios, the packet classification process can be divided into three parts: preparing a packet, executing the lookup algorithm, and processing the packet based on the lookup results [50]. The packet preparation involves extracting the header information. Then, the lookup algorithm is performed based on the header information. Finally, the action associated with the matching rule is executed, such as forwarding or dropping the packet. The core of the entire packet classification process is the lookup algorithm, while the other two parts remain consistent across different methods. Therefore, similar to the open-source implementations of most studies, such as TM [33], HT [37] and DT [34], we mainly measure the lookup time for each algorithm, which intuitively reflects the throughput of packet classification. The average lookup time is compared across

different algorithms. In the experiments, the packet headers are prepared in advance based on the input traffic and the average lookup time is calculated over 10 runs. The matching results are stored in an array for subsequent analysis. The experiments cover synthetic traffic with and without locality, as well as mixed traffic.

a) *Non-local traffic*: Figure 6 shows the average lookup time on non-local traffic of DBT, DT, PT, TT, NM, HT, TM and CS on the 12 rulesets. As we can see, on the scale of both 128k and 256k, DBT has the smallest lookup time on every ruleset compared to the other algorithms, with an average of $0.119\mu s$ and $0.141\mu s$, respectively. This is followed by DT, with an average of $0.181\mu s$ and $0.217\mu s$, respectively. Compared to DT, DBT is 1.51x and 1.53x faster on average, respectively. Compared with PT, TT, NM, HT, TM and CS, DBT is 2.09x, 2.44x, 4.19x, 5.48x, 5.09x and 6.19x faster with 128k rules on average, respectively. For the 256k rulesets, DBT is also 2.22x, 2.7x, 4.78x, 6x, 6.41x and 7.29x faster on average, respectively. In particular, DBT has extremely high lookup performance on some well-separated rulesets due to its concise design. For example, on ACL1, ACL5 and IPC2, the lookup time of DBT is 2.56x, 2.82x and 2.04x faster than DT, respectively. Note that the prediction results of NM contain errors. For instance, across 12 256k rulesets, the average prediction error rate of NM is about 4.34%.

DT significantly decreases the number of constructed tuples, thereby reducing the number of accesses to the hash table and improving lookup performance compared with other algorithms. In contrast, DBT hashes the rules in a single table using DBS in a more efficient manner, simplifying the lookup process and eliminating the requirement to reprocess the packet based on the prefix of the associated tuple. This streamlined approach contributes to superior overall performance compared to DT.

b) *Skewed traffic*: Real Internet traffic often exhibits a significant level of locality [51], [52]. To evaluate the lookup performance of different algorithms under skewed traffic conditions, we employed ClassBench to generate packets with varying degrees of locality based on 12 rulesets. ClassBench generates locality-based header traces using a Pareto distribution, where the parameters α and β determine the degree of skewness. The cumulative distribution function of the Pareto distribution is defined as $D(x) = 1 - (\beta/x)^\alpha$. In our experiments, we set α to 1 and vary β from 0.0001 to 1, resulting in the probability of generating two or more identical packets ranging from 0.005% to 50%.

Skewed traffic contains a high volume of consecutive identical packets, leading to a significant boost in cache hit rates during searches. Figure 7 shows the average lookup time of

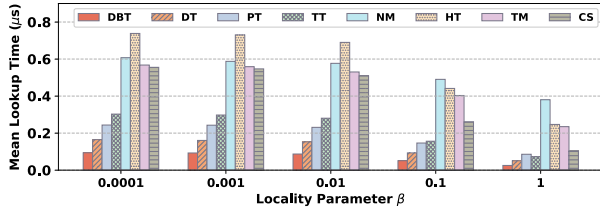


Fig. 7. Lookup time on traffic with different locality.

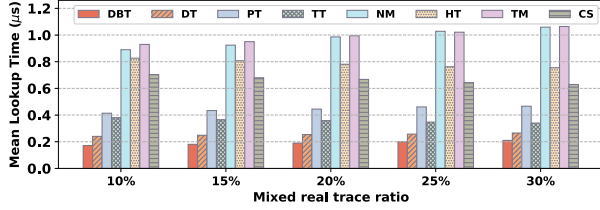


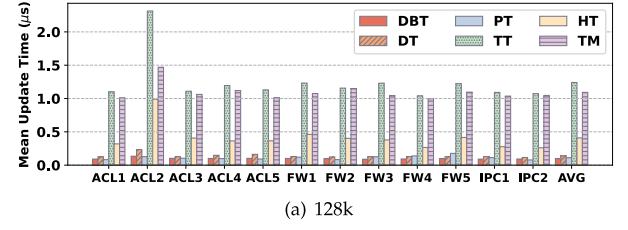
Fig. 8. Lookup time on traffic with varying ratios of real traces.

8 algorithms across 12 256k rulesets when processing traffic with varying locality. It is evident that the lookup performance of all algorithms improves as the locality increases. As β increases from 0.0001 to 1, the lookup time of DBT decreases by 72.97%. In comparison, the lookup times of DT, PT, TT, NM, HT, TM, and CS decrease by 69.32%, 64.8%, 76.1%, 37.37%, 66.57%, 58.62%, and 81.17%, respectively.

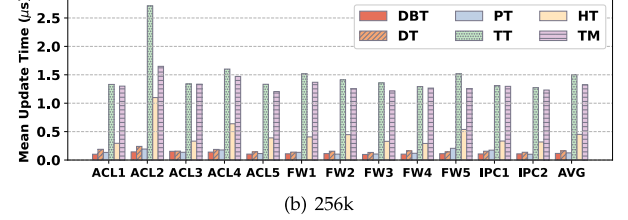
The significant improvement in performance observed in CS can be attributed to its tree structure, which is more cache-sensitive. Frequent accesses to the same branches in CS can significantly boost cache hit rates, leading to enhanced lookup performance. In contrast, NM shows the smallest performance boost due to its heavy reliance on model inference, which involves substantial mathematical computation. Overall, DBT maintains excellent lookup performance under skewed traffic. When β is set to 1, the lookup speed of DBT is 1.99x, 3.38x, 2.85x, 14.88x, 9.73x, 9.25x, and 4.12x than DT, PT, TT, NM, HT, TM, and CS, respectively.

c) Real traffic: Figure 8 shows the average lookup time of all tested algorithms across 12 256k rulesets, processing traffic sets with 10% to 30% real traces. Overall, TSS-based strategies experience performance degradation when handling a large number of non-matching packets, requiring access to all tuples. Conversely, tree-based structures excel at processing such packets, as they can determine non-matching packets at shallower depths.

With an increasing mixed ratio of real traces, the lookup times of DBT, DT, PT, and TM gradually rise. NM, which typically relies on external classifiers in these scenarios, performs similarly to TM. When the mixed ratio rises from 10% to 30%, the lookup times of DBT, DT, PT, TM, and NM increase by 22.46%, 10.48%, 12.67%, 14.45%, and 19%, respectively. In contrast, the lookup times of tree-based strategies TT and CS exhibit a decreasing trend. HT, which employs a multi-level TSS strategy, usually identifies non-matching packets at the first level of the TSS structure, leading to lower lookup times for non-matching packets compared to matching packets. Therefore, HT also shows a decreasing trend in lookup times. The lookup times of TT, CS, and HT decrease



(a) 128k



(b) 256k

Fig. 9. Update time.

by 10.08%, 10.42%, and 8.76%, respectively, as the mixed ratio increases from 10% to 30%.

Although the TSS-based hybrid structure of DBT may lead to a performance decrease when managing traffic with numerous non-matching packets, it still maintains a relatively high level of lookup performance. When the mixed ratio is set to 30%, DBT's average lookup time is reduced by 20.82%, 54.95%, 38.21%, 80.16%, 72.13%, 80.25%, and 66.61% compared to DT, PT, TT, NM, HT, TM, and CS, respectively.

2) Update Time: We test the rule update performance of 6 algorithms since NM and CS cannot support incremental rule updates well. In the experiments, after constructing the data structure, we first randomly delete 5,000 rules from the data structure, and then insert 5,000 new rules. The update time is the average of the deletion and insertion times. As shown in Figure 9, the update speed of DBT is close to that of PT and DT since they only modify one bucket or tuple. Overall, DBT is the fastest since its structure is more concise. In comparison to DT, DBT eliminates the need to access an additional table to determine the corresponding tuple. In contrast, PT requires accessing multiple inner nodes to locate the bucket. The update time of DBT is 10.9% and 28.9% shorter than PT and DT respectively on 128k rulesets on average. On 256k rulesets, the reduction is 7.8% and 28.1%. The update speed of TT and TM is slower since they may rebuild a partial structure (tuple or node). Compared with HT, TM and TT, DBT is 4.12x, 11.01x and 12.51x faster on 128k rulesets and 3.88x, 11.37x and 12.91x faster on 256k rulesets, respectively.

3) Construction Time: As shown in Figure 10, the average construction time of DBT is larger than TT and TM since DBT needs to extract DBS. However, DBT only extracts DBS once, therefore the overall construction time is comparable, with an average of 1.24s and 2.53s on the 128k and 256k rulesets, respectively. NM and HT require model training, so their construction time is more than tens of minutes, far greater than the other algorithms. PT and DT need to search for the best configuration, so their construction time is also longer.

4) Memory Footprint: Figure 11 shows the memory footprint of the eight algorithms, excluding the memory footprint of the rules themselves. Of these, the memory footprint of

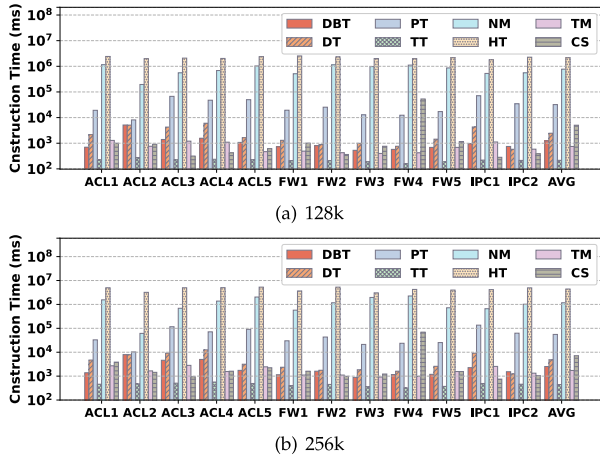


Fig. 10. Construction time.

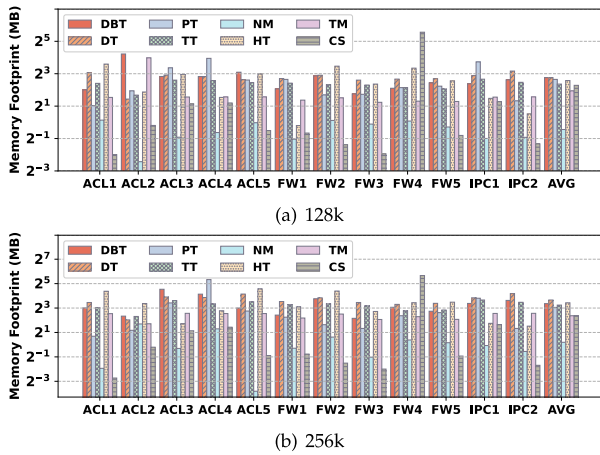


Fig. 11. Memory footprint.

NM is much smaller than the others, with an average of 0.738MB and 1.14MB on 128k and 256k rulesets, respectively. The reason for this is that NM transforms the ruleset into a model which occupies very little memory. The memory footprint of DBT is similar to other traditional methods. On the rulesets of size 256k, DBT occupies 10.26MB memory on average. In contrast, DT, PT, TT, HT, TM and CS consume 12.67MB, 8.32MB, 9.46MB, 10.79MB, 5.18MB and 5.19MB respectively on average. TM and CS demonstrate low memory costs due to their limited ability to effectively separate rulesets, resulting in fewer indexes. However, their lookup performances are comparatively inferior when compared to other methods.

E. Dynamic Experiments

In this section, we design experiments to simulate a dynamic update scenario and evaluate the efficacy of the dynamic reconfiguration strategy of DBT. We set the traffic window size to 1,000,000 packets and. In each window, we delete 10,000 rules and insert 10,000 new rules. The experiment utilizes five 256k rulesets: ACL1-ACL5. Initially, we construct the index structure based on ACL1 and subsequently insert new rules from ACL2 to ACL5 in the first four windows. Thereafter, we continue inserting rules from ACL5 until all rules were integrated into the index structure.

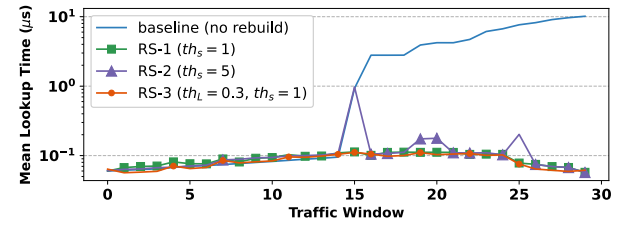


Fig. 12. The lookup performance in the dynamic scenario with different reconfiguration strategies.

Figure 12 shows the average lookup time of each window with different reconfiguration strategies (RS). We use the results without rebuilding the index structure as the baseline. The two approaches, RS-1 and RS-2, re-extract the DBS in each window, with threshold th_s set to 1 and 5, respectively. RS-3 re-extracts the DBS based on the threshold th_L set to 0.3, with th_s set to 1. We mark the windows where reconstruction takes place using distinct markers.

DBT demonstrates the ability to sustain stable performance while requiring only a small number of updates to rules. After inserting new rules from different rulesets over the first four windows without rebuilding, the DBT lookup time increases by 10.97% compared to the first window. For RS-1, frequent rebuilding occurs due to the smaller th_s , yet the lookup times in each window remain similar to the baseline, suggesting that minor variations in DBS have minimal impact on lookup performance. However, as the number of rule updates increases, the partitioning effect of original DBS decreases. As seen in window 15, the lookup time significantly increases.

For RS-2, although increasing th_s to 5 reduces the frequency of reconstruction to 11, the looser constraint leads to higher lookup times in certain windows, such as windows 15 and 25. In comparison, RS-3 introduces th_L to control the timing of DBS re-extraction, reducing the number of reconstructions from 27 in RS-1 to 9, while also mitigating the delayed adjustments observed in RS-2. In conclusion, by combining thresholds th_L for triggering DBS re-extraction and th_s for reconstruction, DBT can maintain high lookup performance while reducing resource wastage.

F. Real Application

To confirm the practicality of DBT in real-world applications, we evaluate the packet classification throughput of DBT using the DPDK performance test tool and compare it with the `rte_acl` classifier provided in DPDK. The classification algorithm utilized in `rte_acl` is a multi-bit trie with a stride of 8. In the experiment, we utilize 12 256k ruleset and continuously transmit randomly generated packets.

As illustrated in Figure 13, the throughput of DBT significantly surpasses that of `rte_acl` across all rulesets, with an average increase of 4.37x. Specifically, on ACL1 and ACL5, the throughput of DBT is 12.6x and 9.7x higher than `rte_acl`, respectively, indicating its superior performance on evenly distributed rulesets. Moreover, while `rte_acl` exhibits stable performance across different rulesets, it consumes more memory, averaging 4.66GB across 12 rulesets. Overall, DBT exhibits significant advantages over `rte_acl` in both packet

TABLE V
STATISTICAL RESULTS OF DBTABLE

Ruleset	A. <i>Skew</i>	E. Bits	UB	EB	TSN	ATN	MBS	MTN	AAB	AAT	AAR	MAB	MAT	MAR
ACL1	0.15	17	84.83%	86.24%	1	9	29	9	1.35	0.36	1.81	2	9	62
ACL2	0.79	15	40.46%	62.91%	1	31	46	31	1.32	10.06	14.39	2	31	127
ACL3	0.22	17	45.78%	93.31%	525	1.01	51	10	1.34	1.09	4.44	2	11	152
ACL4	0.57	17	48.08%	88.78%	371	3.95	39	14	1.39	2.04	4.65	2	21	115
ACL5	0.16	17	88.27%	87.5%	1	1	10	1	1.01	0.01	1.91	2	1	10
FW1	0.42	16	84.59%	95.9%	47	4.38	24	19	1.95	3.27	4.13	2	21	151
FW2	0.49	17	72.31%	96.29%	1	6	10	6	1.71	1.16	3.22	2	6	23
FW3	0.36	16	95.21%	97.29%	1	19	49	19	1.92	1.9	3.59	2	19	167
FW4	0.46	16	68.95%	88.83%	1	11	53	11	1.71	1.68	7.18	2	11	218
FW5	0.47	16	79.25%	96.28%	68	2.75	38	17	1.95	3.59	4.99	2	20	181
IPC1	0.39	17	85.91%	97.29%	1	7	15	7	1.61	1.08	2.96	2	7	57
IPC2	0.37	17	80.92%	99.99%	1	1	2	1	1.62	0.62	1.57	2	1	3

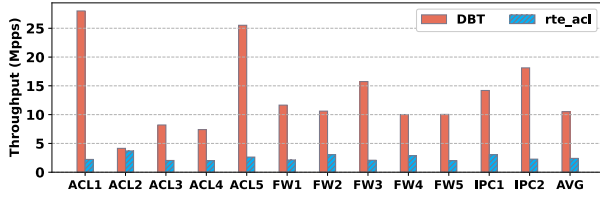


Fig. 13. The throughput of DBTable compared with the implementation in DPDK *rte_acl* based on the DPDK performance test tool.

classification throughput and memory footprint, highlighting its potential for real-world applications.

G. Analysis

This section provides a comprehensive overview of DBT to elucidate the factors contributing to its high performance. Table V presents the statistical details of DBT for 12 rulesets of size 256k. The table encompasses four main aspects: the number of extracted bits, structural information, average access count of the structure, and worst-case information. Given that the default setting of *binth* is 4, 256k rulesets necessitate a minimum of 16 bits to evenly partition the rules and satisfy the threshold *binth*. Therefore, the efficiency of the extracted DBS increases as its size approaches 16.

As shown in Table V, the maximum number of extracted bits across the 12 rulesets is 17, highlighting the efficiency of our iterative DBS extraction method. When considering the ACL2 ruleset, exclusively relying on a one-dimensional feature to partition the ruleset proves challenging due to significant skewness in the ruleset, as evident in the second column of Table V. To mitigate the utilization of invalid or inefficient bits, the extraction algorithm terminates prematurely, resulting in a DBS size of only 15 bits. We adopt the metric *Skew*, as defined in [42] and [53], to quantify the degree of ruleset skewness, which is formulated as:

$$Skew = 1 - \frac{weight(light)}{weight(heavy)} \quad (3)$$

Of these, *weight()* counts the number of rules that share identical prefixes in the IP field. *heavy* and *light* respectively represent nodes with a higher number of rules and nodes with fewer rules. Table V shows the average value of *Skew* for all SIP and DIP prefixes. The degree of skewness serves as an indicator of the challenges associated with partitioning the ruleset, thereby directly impacting lookup performance.

1) *Structure Information*: In Table V, we present the utilization rate of buckets (UB, rate of buckets containing rules), the expected bucket rate (EB, rate of buckets with sizes less than *binth*), the number of constructed TSS structures (TSN), the average number of tuples of the constructed TSS structures (ATN), the maximum size of buckets (MBS), and the maximum number of tuples across all TSS structures (MTN). It is evident that for the majority of rulesets, the UB surpasses 70%, affirming the effectiveness of the first index structure of DBT and ensuring the memory usage of DBT. EB serves as a visual indicator of DBT's ability to partition rules. With the exception of ACL2, the EB exceeds 85% for all rulesets, and even reaches 99.99% for IPC2, highlighting the efficacy of DBT's segregation.

TSN represents the number of TSS structures constructed in DBT. Due to the effective separation of most rules, DBT only constructs a single substructure for the wildcard bucket, which tends to accumulate rules in the majority of rulesets. However, for ACL3 and ACL4, additional substructures are constructed due to the rule aggregation within specific intervals. ATN indicates the average number of tuples within the constructed TSS structure. It can be seen that the utilization of DBS separation leads to a smaller average number of constructed tuples, thereby mitigating prolonged lookup time in the substructure. Lastly, we provide the maximum number of rules contained in buckets and the maximum number of constructed tuples, both of which directly impact worst-case performance. Given the average MBS and MTN values of 30.5 and 12.8 respectively across the 12 rulesets, DBT ensures satisfactory overall performance.

2) *Average Access Count*: The average access count reflects the overall lookup performance of DBT, encompassing the average number of accessed first-layer buckets (AAB), tuples (AAT), and rules (AAR). As shown in Table V, the average values for AAB, AAT and AAR across the 12 rulesets are 1.57, 2.24 and 4.57 respectively, affirming the fulfillment of our design objectives by DBT. Notably, for ACL5, DBT primarily requires access to a single bucket, resulting in superior lookup performance compared to other rulesets.

3) *Worst-Case Information*: The maximum number of accessed first-layer buckets (MAB), the tuples (MAT), and the rules (MAR) collectively serve as comprehensive indicators of the worst-case performance exhibited by DBT. As discussed in Section IV-E, DBT accesses at most two buckets in the first

layer, while the maximum number of accessed tuples depends on the total number of constructed tuples. Thus, in the worst-case scenario, the mean values for MAB and MAT across the 12 rulesets are 2 and 13.17 respectively, as guaranteed by the structural information presented in Table V. Furthermore, the skewness-aware hybrid structure of DBT ensures the guaranteed number of rules accessed in the worst-case, with an average of 105.5 across the 12 rulesets.

In conclusion, the statistics provided in Table V further underscore the concise yet efficient design of DBT. It achieves exceptional overall performance while ensuring worst-case performance.

VI. CONCLUSION

In this paper, we present DBTable, a novel structure designed to achieve high lookup performance while ensuring fast update speeds. DBT constructs a hash table using an extracted discriminative bitset (DBS), therefore enabling the majority of update operations to require only a single memory access. Additionally, we propose an iterative method for extracting a DBS to evenly partition the rules, a critical aspect for achieving superior lookup performance. Furthermore, we optimize the worst-case lookup performance by incorporating a skewness-aware hybrid structure. The experiment results show that DBT surpasses state-of-the-art algorithms, exhibiting significant improvements in overall lookup speed ranging from 1.53x to 7.29x while concurrently maintaining the fastest update performance.

REFERENCES

- [1] D. E. Taylor, "Survey and taxonomy of packet classification techniques," *ACM Comput. Surveys*, vol. 37, no. 3, pp. 238–275, Sep. 2005.
- [2] B. Nagpal, N. Singh, N. Chauhan, and R. Murari, "A survey and taxonomy of various packet classification algorithms," in *Proc. Int. Conf. Adv. Comput. Eng. Appl.*, Mar. 2015, pp. 8–13.
- [3] T. Y. C. Woo, "A modular approach to packet classification: Algorithms and results," in *Proc. IEEE INFOCOM. Conf. Comput. Commun.*, vol. 3, Jun. 2000, pp. 1213–1222.
- [4] A. Hakiri, A. Gokhale, P. Berthou, D. C. Schmidt, and T. Gayraud, "Software-defined networking: Challenges and research opportunities for future Internet," *Comput. Netw.*, vol. 75, pp. 453–471, Dec. 2014.
- [5] Q. Yan, F. R. Yu, Q. Gong, and J. Li, "Software-defined networking (SDN) and distributed denial of service (DDoS) attacks in cloud computing environments: A survey, some research issues, and challenges," *IEEE Commun. Surveys Tuts.*, vol. 18, no. 1, pp. 602–622, 1st Quart., 2016.
- [6] R. Amin, M. Reisslein, and N. Shah, "Hybrid SDN networks: A survey of existing approaches," *IEEE Commun. Surveys Tuts.*, vol. 20, no. 4, pp. 3259–3306, 4th Quart., 2018.
- [7] R. Deb and S. Roy, "A comprehensive survey of vulnerability and information security in SDN," *Comput. Netw.*, vol. 206, Apr. 2022, Art. no. 108802.
- [8] F. Baboescu and G. Varghese, "Scalable packet classification," *IEEE/ACM Trans. Netw.*, vol. 13, no. 1, pp. 2–14, Feb. 2005.
- [9] A. Matsushita, M. Nakamura, F. Hamaoka, and S. Okamoto, "Transmission technologies beyond 400-Gbps/carrier employing high-order modulation formats," in *Proc. Eur. Conf. Opt. Commun. (ECOC)*, Sep. 2018, pp. 1–3.
- [10] Y. Chen, W. Liu, Z. Niu, Z. Feng, Q. Hu, and T. Jiang, "Pervasive intelligent endogenous 6G wireless systems: Prospects, theories and key technologies," *Digit. Commun. Netw.*, vol. 6, no. 3, pp. 312–320, Aug. 2020.
- [11] C. R. Meiners, A. X. Liu, and E. Torng, *Hardware Based Packet Classification for High Speed Internet Routers*. Cham, Switzerland: Springer, 5555.
- [12] B. Vamanan and T. N. Vijaykumar, "TreeCAM: Decoupling updates and lookups in packet classification," in *Proc. 7th Conf. Emerg. Netw. Exp. Technol.*, Dec. 2011, pp. 1–12.
- [13] O. Rottenstreich, R. Cohen, D. Raz, and I. Keslassy, "Exact worst case TCAM rule expansion," *IEEE Trans. Comput.*, vol. 62, no. 6, pp. 1127–1140, Jun. 2013.
- [14] O. Rottenstreich and J. Tapolcai, "Lossy compression of packet classifiers," in *Proc. ACM/IEEE Symp. Archit. Netw. Commun. Sys. (ANCS)*, 2015, pp. 39–50.
- [15] O. Rottenstreich, I. Keslassy, A. Hassidim, H. Kaplan, and E. Porat, "Optimal in/out TCAM encodings of ranges," *IEEE/ACM Trans. Netw.*, vol. 24, no. 1, pp. 555–568, Feb. 2016.
- [16] O. Rottenstreich and J. Tapolcai, "Optimal rule caching and lossy compression for longest prefix matching," *IEEE/ACM Trans. Netw.*, vol. 25, no. 2, pp. 864–878, Apr. 2017.
- [17] K. Kogan, S. I. Nikolenko, O. Rottenstreich, W. Culhane, and P. Eugster, "Exploiting order independence for scalable and expressive packet classification," *IEEE/ACM Trans. Netw.*, vol. 24, no. 2, pp. 1251–1264, Apr. 2016.
- [18] K. Kogan, S. I. Nikolenko, P. Eugster, A. Shalimov, and O. Rottenstreich, "Efficient FIB representations on distributed platforms," *IEEE/ACM Trans. Netw.*, vol. 25, no. 6, pp. 3309–3322, Dec. 2017.
- [19] P. Gupta and N. McKeown, "Classifying packets with hierarchical intelligent cuttings," *IEEE Micro*, vol. 20, no. 1, pp. 34–41, 2000.
- [20] S. Singh, F. Baboescu, G. Varghese, and J. Wang, "Packet classification using multidimensional cutting," in *Proc. Conf. Appl. Technol., Archit., Protocols Comput. Commun.*, Aug. 2003, pp. 213–224.
- [21] Y. Qi, L. Xu, B. Yang, Y. Xue, and J. Li, "Packet classification algorithms: From theory to practice," in *Proc. IEEE INFOCOM*, Apr. 2009, pp. 648–656.
- [22] B. Vamanan, G. Voskuilen, and T. N. Vijaykumar, "EffiCuts: Optimizing packet classification for memory and throughput," in *Proc. ACM SIGCOMM Conf.*, Aug. 2010, pp. 207–218.
- [23] J. Fong, X. Wang, Y. Qi, J. Li, and W. Jiang, "ParaSplit: A scalable architecture on FPGA for terabit packet classification," in *Proc. IEEE 20th Annu. Symp. High-Perform. Interconnects*, Aug. 2012, pp. 1–8.
- [24] W. Li and X. Li, "HybridCuts: A scheme combining decomposition and cutting for packet classification," in *Proc. IEEE 21st Annu. Symp. High-Perform. Interconnects*, Aug. 2013, pp. 41–48.
- [25] P. He, G. Xie, K. Salamatian, and L. Mathy, "Meta-algorithms for software-based packet classification," in *Proc. IEEE 22nd Int. Conf. Netw. Protocols*, Oct. 2014, pp. 308–319.
- [26] S. Yingchareonthawornchai, J. Daly, A. X. Liu, and E. Torng, "A sorted partitioning approach to high-speed and fast-update OpenFlow classification," in *Proc. IEEE 24th Int. Conf. Netw. Protocols (ICNP)*, Nov. 2016, pp. 1–10.
- [27] Z. Liu, S. Sun, H. Zhu, J. Gao, and J. Li, "BitCuts: A fast packet classification algorithm using bit-level cutting," *Comput. Commun.*, vol. 109, pp. 38–52, Sep. 2017.
- [28] J. Daly and E. Torng, "ByteCuts: Fast packet classification by interior bit extraction," in *Proc. IEEE INFOCOM Conf. Comput. Commun.*, Apr. 2018, pp. 2654–2662.
- [29] W. Li, T. Yang, Y. Chang, T. Li, and H. Li, "TabTree: A TSS-assisted bit-selecting tree scheme for packet classification with balanced rule mapping," in *Proc. ACM/IEEE Symp. Archit. Netw. Commun. Syst. (ANCS)*, Sep. 2019, pp. 1–8.
- [30] W. Li, X. Li, H. Li, and G. Xie, "CutSplit: A decision-tree combining cutting and splitting for scalable packet classification," in *Proc. IEEE INFOCOM Conf. Comput. Commun.*, Apr. 2018, pp. 2645–2653.
- [31] V. Srinivasan, S. Suri, and G. Varghese, "Packet classification using tuple space search," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 29, no. 4, pp. 135–146, Oct. 1999.
- [32] B. Pfaff et al., "The design and implementation of open vSwitch," in *Proc. 12th USENIX Sympo. Netw. Sys. Design Implement.*, May 2015, pp. 117–130.
- [33] J. Daly et al., "TupleMerge: Fast software packet processing for online packet classification," *IEEE/ACM Trans. Netw.*, vol. 27, no. 4, pp. 1417–1431, Aug. 2019.
- [34] C. Zhang, G. Xie, and X. Wang, "DynamicTuple: The dynamic adaptive tuple for high-performance packet classification," *Comput. Netw.*, vol. 202, Jan. 2022, Art. no. 108630.
- [35] E. Liang, H. Zhu, X. Jin, and I. Stoica, "Neural packet classification," in *Proc. ACM Special Interest Group Data Commun.*, Aug. 2019, pp. 256–269.

- [36] A. Rashelbach, O. Rottenstreich, and M. Silberstein, "A computational approach to packet classification," *IEEE/ACM Trans. Netw.*, vol. 30, no. 3, pp. 1073–1087, Jun. 2022.
- [37] Y. Liu et al., "HybridTSS: A recursive scheme combining coarse- and fine-grained tuples for packet classification," in *Proc. 6th Asia-Pacific Workshop Netw.*, Jul. 2022, pp. 1–7.
- [38] W. Li et al., "Tuple space assisted packet classification with high performance on both search and update," *IEEE J. Sel. Areas Commun.*, vol. 38, no. 7, pp. 1555–1569, Jul. 2020.
- [39] J. Zhong, Z. Wei, S. Zhao, and S. Chen, "TupleTree: A high-performance packet classification algorithm supporting fast rule-set updates," *IEEE/ACM Trans. Netw.*, vol. 1, no. 1, pp. 1–15, May 2022.
- [40] Z. Liao et al., "PT-tree: A cascading prefix tuple tree for packet classification in dynamic scenarios," *IEEE/ACM Trans. Netw.*, vol. 1, no. 1, pp. 1–14, May 2023.
- [41] N. McKeown et al., "OpenFlow: Enabling innovation in campus networks," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 38, no. 2, pp. 69–74, Mar. 2008.
- [42] J. Matousek, A. Lucanský, D. Janecek, J. Sabo, J. Korenek, and G. Antichi, "ClassBench-ng: Benchmarking packet classification algorithms in the OpenFlow era," *IEEE/ACM Trans. Netw.*, vol. 30, no. 5, pp. 1912–1925, Oct. 2022.
- [43] M. H. Overmars and F. A. van der Stappen, "Range searching and point location among fat objects," *J. Algorithms*, vol. 21, no. 3, pp. 629–656, Nov. 1996.
- [44] T. Kraska, A. Beutel, E. H. Chi, J. Dean, and N. Polyzotis, "The case for learned index structures," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, May 2018, pp. 489–504.
- [45] G. T. Reddy et al., "Analysis of dimensionality reduction techniques on big data," *IEEE Access*, vol. 8, pp. 54776–54788, 2020.
- [46] L. Askari, P. Majidzadeh, O. Ayoub, and M. Tornatore, "Exploiting DPDK in containerized environment with unsupported hardware," in *Proc. IEEE Conf. Netw. Function Virtualization Softw. Defined Netw.*, Nov. 2020, pp. 7–12.
- [47] A. Belkhiri, M. Pepin, M. Bly, and M. Dagenais, "Performance analysis of DPDK-based applications through tracing," *J. Parallel Distrib. Comput.*, vol. 173, pp. 1–19, Mar. 2023.
- [48] H. Hamed, A. El-Atawy, and E. Al-Shaer, "Adaptive statistical optimization techniques for firewall packet filtering," in *Proc. IEEE INFOCOM. 25TH IEEE Int. Conf. Comput. Commun.*, Jun. 2006, pp. 1–12.
- [49] Z. Trabelsi, L. Zhang, S. Zeidan, and K. Ghoudi, "Dynamic traffic awareness statistical model for firewall performance enhancement," *Comput. Secur.*, vol. 39, pp. 160–172, Nov. 2013.
- [50] M. Finsterbusch, C. Richter, E. Rocha, J.-A. Müller, and K. Hanssgen, "A survey of payload-based traffic classification approaches," *Proc. IEEE Commun. Surv. Tutor.*, vol. 16, no. 2, pp. 1135–1156, Jun. 2014.
- [51] N. Sarrar, S. Uhlig, A. Feldmann, R. Sherwood, and X. Huang, "Leveraging Zipf's law for traffic offloading," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 42, no. 1, pp. 16–22, Jan. 2012.
- [52] M. Kassim and N. A. Ayop, "Adaptive policing algorithms on inbound Internet traffic using generalized Pareto model," in *Proc. 11th Int. Conf. Internet Technol. Secured Trans. (ICITST)*, Dec. 2016, pp. 217–222.
- [53] D. E. Taylor and J. S. Turner, "ClassBench: A packet classification benchmark," *IEEE/ACM Trans. Netw.*, vol. 15, no. 3, pp. 499–511, Jun. 2007.



Shiyong Qian (Member, IEEE) received the Ph.D. degree in computer science from Shanghai Jiao Tong University in 2015. He is currently an Associate Researcher with the Department of Computer Science and Engineering, Shanghai Jiao Tong University. His research interests include event matching for content-based publish/subscribe systems, resource scheduling for the hybrid cloud, and driving recommendation with vehicular networks.



Zhonglong Zheng (Member, IEEE) received the B.Sc. degree from China University of Petroleum, China, in 1999, and the Ph.D. degree from Shanghai Jiao Tong University, China, in 2005. He is currently a Full Professor with the College of Mathematics and Computer Science, Zhejiang Normal University, China. His research interests include machine learning, computer vision, and blockchain.



Jiange Zhang received the Ph.D. degree in computer science from the University of Colorado at Colorado Springs in 2020. He is currently a Principal Engineer at Huawei. His research interests include concurrent programming and network routing.



Jian Cao (Senior Member, IEEE) received the Ph.D. degree from Nanjing University of Science and Technology in 2000. He is currently a Professor with the Department of Computer Science and Engineering, Shanghai Jiao Tong University. His main research interests include service computing, network computing, and intelligent data analytics.



Guangtao Xue (Member, IEEE) received the Ph.D. degree from the Department of Computer Science and Engineering, Shanghai Jiao Tong University, China, in 2004. He is currently a Professor with the Department of Computer Science and Engineering, Shanghai Jiao Tong University. His research interests include vehicular ad hoc networks, wireless networks, mobile computing, and distributed computing. He is a member of the IEEE Computer Society and the IEEE Communication Society.



Zhengyu Liao is currently pursuing the master's degree with the College of Mathematics and Computer Science, Zhejiang Normal University, China. His research interests include matching algorithms of publish/subscribe systems and network packet classification.



Minglu Li (Fellow, IEEE) received the Ph.D. degree from Shanghai Jiao Tong University in 1996. He is currently a Distinguished Professor with the College of Mathematics and Computer Science, Zhejiang Normal University, and a Professor with the Department of Computer Science and Engineering, Shanghai Jiao Tong University. His main research interests include cloud computing, image processing, and e-commerce.